IaC

Infrastructure as Code Challenge

Recreate Lab 1 IAM resources using Terraform pointed at LocalStack.

Terraform · LocalStack · IAM · Least Privilege

IaC = Infrastructure as Code

Treat infrastructure like software - written, reviewed, versioned, reused, and destroyed safely.

Manual CLI

Create group

Attach policy

Create user

Add user to group

Good for learning mechanics.
Harder to repeat consistently.

convert into

IaC

main.tf

Desired state is declared once

Tool figures out the changes

Same result can be recreated

Declarative

Describe what should exist.

Idempotent

Run again → no unnecessary change.

State

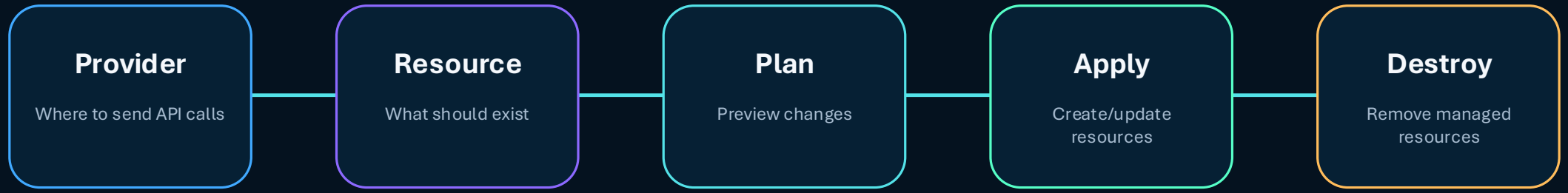
Tool remembers what it manages.

Versioned

Track infra changes in Git.

Terraform

Terraform is an IaC tool. In this lab, it sends AWS-style API calls to LocalStack.



Code

```
resource "aws_iam_group" "admins" {
  name = "Admins"
}


resource "aws_iam_user" "cloud_admin" {
  name = "CloudAdmin_<YOURNAME>"
}
```

Explanation

Terraform does not care about the button you clicked.

It cares whether the declared infrastructure already exists.

Terraform Command	What It Does
`terraform init`	Downloads and initializes the required Terraform providers, such as the **AWS provider**.
`terraform fmt`	Formats the Terraform code into a consistent, readable style.
`terraform validate`	Checks whether the Terraform configuration is **syntactically valid**.
`terraform plan`	Shows **what Terraform intends to create, change, or destroy** before making changes.
`terraform apply`	**Performs the planned actions** and creates or modifies the infrastructure.
`terraform destroy`	**Removes the infrastructure** managed and created by Terraform.

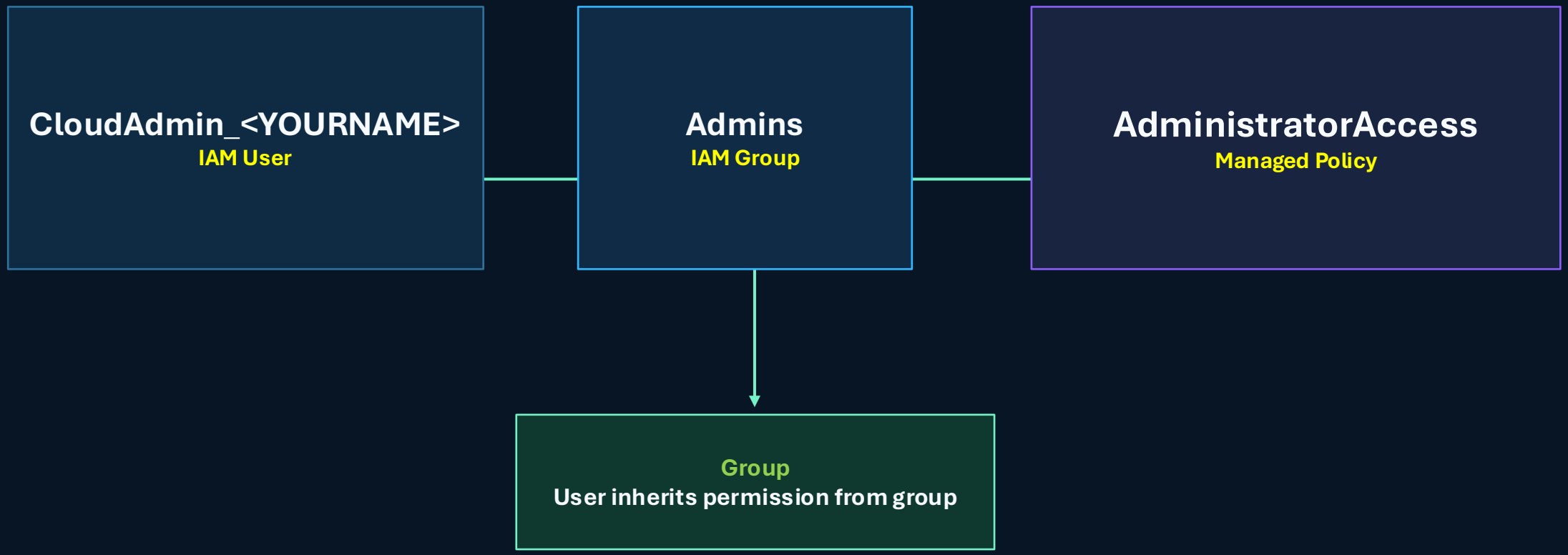
From manual commands to repeatable infrastructure

The goal is not “learn Terraform syntax.”
The goal is to change how we build cloud infrastructure.

Manual CLI teaches mechanics	create group · attach policy · create user · add user to group
IaC teaches operating model	define desired state · review plan · apply safely · verify independently
Security lesson stays the same	permissions flow through groups, not directly to individual users

Target state: the same IAM design, but coded

Students recreate the least-privilege admin setup from Task 2 using Terraform.



Create this with Terraform resources, not manual AWS CLI commands.

Terraform talks to LocalStack

main.tf → AWS Provider → LocalStack localhost:4566 → IAM Resources

The Terraform file has two jobs

Tell Terraform where to send AWS API calls

```
provider "aws" {
  access_key = "test"
  secret_key = "test"
  region     = "us-east-1"

  endpoints {
    iam = "http://localhost:4566"
    sts = "http://localhost:4566"
  }

  skip_credentials_validation = true
  skip_metadata_api_check     = true
  skip_requesting_account_id  = true
}
```

1

Declare the IAM resources that must exist

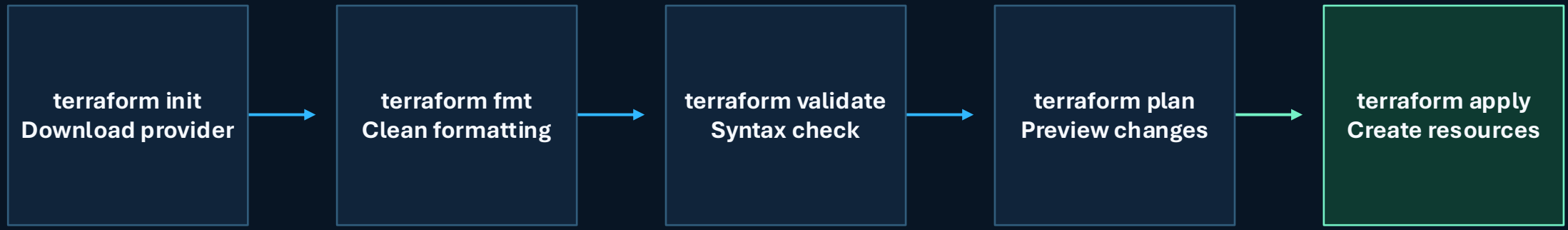
```
resource "aws_iam_group" "admins" {
  name = "Admins"
}

resource "aws_iam_user" "cloud_admin" {
  name = "CloudAdmin_YOURNAME"
}

resource "aws_iam_group_policy_attachment" "admin_policy" {
  group      = aws_iam_group.admins.name
  policy_arn = "arn:aws:iam::aws:policy/AdministratorAccess"
}
```

2

The command sequence matters



Pause at terraform plan

A good engineer reviews what will change before applying it.

Do not trust “Apply complete” blindly

Students must verify through AWS CLI pointed at LocalStack.

```
EP='--endpoint-url=http://localhost:4566'

aws $EP iam get-group \
 --group-name Admins

aws $EP iam list-attached-group-policies \
 --group-name Admins
```

- Confirm CloudAdmin_<YOURNAME> appears as a group member.
- Confirm AdministratorAccess is attached to the Admins group.
- Screenshot both outputs for submission.

Terraform compares desired state vs current state

terraform plan

No changes.
Your infrastructure matches the configuration.

Change username in main.tf

terraform plan

Terraform shows the difference.

This is why IaC scales better than manual clicks and one-off terminal commands.

What need to submit

01	**main.tf**	Terraform configuration that creates IAM group, user, policy attachment, and membership.
02	**Command evidence**	Screenshots of init, validate, plan, apply, and destroy.
03	**Verification evidence**	AWS CLI output proving group membership and attached policy.
04	**Short reflection**	Explain why IaC is better than repeating manual commands.

Final cleanup:
terraform destroy

Your task: convert Task 2 into IaC

Create it.
Plan it.
Apply it.
Verify it.
Destroy it.

Required resources:

1. aws_iam_group
2. aws_iam_user
3. aws_iam_group_policy_attachment
4. aws_iam_user_group_membership

Question to answer

Why is Terraform considered Infrastructure as Code, and what advantage does it provide compared with manual AWS CLI commands?

Constraint: point Terraform to LocalStack at http://localhost:4566.

MAKE SURE TO INSTALL TERRAFORM

OS	Installation
macOS	`brew tap hashicorp/tap` `brew install hashicorp/tap/terraform`
Windows	`winget install Hashicorp.Terraform`
Linux (Ubuntu/Debian)	Install Terraform from HashiCorp's official APT repository

terraform --version

Install Terraform

↓

terraform --version

↓

terraform init

↓

terraform fmt

↓

terraform validate

↓

terraform plan

↓

terraform apply

↓

Verify resources

```
EP='--endpoint-url=http://localhost:4566'

aws $EP iam get-group \
 --group-name Admins-1

aws $EP iam list-attached-group-policies \
 --group-name Admins-1
```

↓

terraform destroy

For your lab, you usually run it when:

- you want to reset and try again cleanly
- you made a mistake and want to recreate the resources
- you finished the lab and want to clean up
- you want your Terraform state to match an empty environment again

Resource #1 — IAM Group

CODE

```
aws $EP iam create-group \
        --group-name Admins
```

TERRAFOAM

```
resource "aws_iam_group" "admins" {
name = "Admins"
}
```

Resource #2 — Policy attachment

TERRAFOAM

```
resource "aws_iam_group_policy_attachment" "admin_policy" {
  group = aws_iam_group.admins.name
  policy_arn = "arn:aws:iam::aws:policy/AdministratorAccess"
}
```

CODE

```
aws $EP iam attach-group-policy \
--group-name Admins \
--policy-arn arn:aws:iam::aws:policy/AdministratorAccess
```

Resource #3 — IAM User

TERRAFOAM

```
resource "aws_iam_user" "cloud_admin" {
   name = "CloudAdmin_DANI"
}
```

CODE

```
aws $EP iam create-user \
    --user-name CloudAdmin_DANI
```

Resource #4 — Group membership

TERRAFOAM

```
resource "aws_iam_user_group_membership" "cloud_admin_membership" {

  user = aws_iam_user.cloud_admin.name

  groups = [
            aws_iam_group.admins.name
   ]
}
```

CODE

```
aws $EP iam add-user-to-group \
        --group-name Admins \
        --user-name CloudAdmin_DANI
```

main.tf

```
terraform {
 required_providers {
  aws = {
   source = "hashicorp/aws"
  }
 }
}

# ------------------------------------------
# Point Terraform to LocalStack
# ------------------------------------------

provider "aws" {
 access_key = "test"
 secret_key = "test"
 region     = "us-east-1"

 # We are using LocalStack, NOT real AWS
 endpoints {
  iam = "http://localhost:4566"
  sts = "http://localhost:4566"
 }

 skip_credentials_validation = true
 skip_metadata_api_check     = true
 skip_requesting_account_id  = true
}

# ------------------------------------------
# 1. Create IAM Group
# ------------------------------------------

resource "aws_iam_group" "admins" {
 name = "Admins"
}

# ------------------------------------------
# 2. Attach AdministratorAccess Policy to the GROUP
# ------------------------------------------

resource "aws_iam_group_policy_attachment" "admin_policy" {
 group      = aws_iam_group.admins.name
 policy_arn = "arn:aws:iam::aws:policy/AdministratorAccess"
}

# ------------------------------------------
# 3. Create IAM User
# ------------------------------------------

resource "aws_iam_user" "cloud_admin" {
 name = "CloudAdmin_DANI"
}

# ------------------------------------------
# 4. Add User to Admins Group
# ------------------------------------------
resource "aws_iam_user_group_membership" "cloud_admin_membership" {
 user = aws_iam_user.cloud_admin.name

 groups = [
  aws_iam_group.admins.name
 ]
}
```